Solution: Kth Smallest Element in a BST (medium)

Problem Statement

Given a `root` node of the Binary Search Tree (BST) and integer 'k'. Return the Kth smallest element among all node values of the binary tree.

Examples:

1. **Example 1**:

   Input:

   ```
      8
     / \
    3   10
   / \    \
  1   6    14
     /  \  /
    4   7 13
   ```

   k = 4

   Expected Output: 6

   Justification: The in-order traversal of the tree is [1, 3, 4, 6, 7, 8, 10, 13, 14]. The 4th element in this sequence is 6.

2. **Example 2**:

   Input:

   ```
      5
     / \
    2   6
   /
  1
   ```

   k = 3

   Expected Output: 5

   Justification: The in-order traversal of the tree is [1, 2, 5, 6]. The 3rd element in this sequence is 5.

3. **Example 3**:

   Input:

```
1
 \
  3
 /
2
```

k = 2

Expected Output: 2

Justification: The in-order traversal of the tree is [1, 2, 3]. The 2nd element in this sequence is 2.

Solution

The most intuitive approach to this problem is to perform an in-order traversal on the BST. Given the nature of BSTs, an in-order traversal will result in a sequence of nodes in ascending order. Thus, by collecting the nodes in a list during traversal, the kth element of this list would be our answer.

However, there is a more efficient approach. We don't necessarily need to traverse the entire BST. We can halt our in-order traversal as soon as we've visited the kth node. This is achieved by using a counter that's incremented every time we visit a node during our traversal. When the counter matches 'k', we've found our kth smallest element.

Algorithm Walkthrough:
Given the tree from Example 1:

```
      8
     / \
    3   10
   / \    \
  1   6    14
     /  \  /
    4   7  13
```

With k = 4, the steps are:

- Start at the root (8). Since there's a left child, move to the left child (3).
- From node 3, again move to its left child (1). It's the leftmost node, so count it as the first smallest element.
- Move up to node 3. It's the second smallest.
- Now move to its right child (6). Since 6 also has a left child, move to that (4). It's the third smallest.
- Move up to node 6. It's the fourth smallest - which is what we are looking for. Hence, we stop and return 6.

Here is the visual representation of the algorithm:

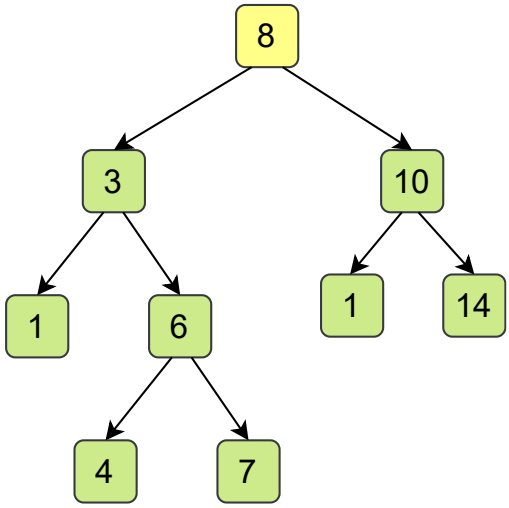
k=4

We will perform an inorder traversal(Left, Root, Right) and keep tracking the smallest element by incrementing the count and as soon as the count==k, we will store that node value in a variable

We will start from the root 8 and will keep going to its left until there is no further node

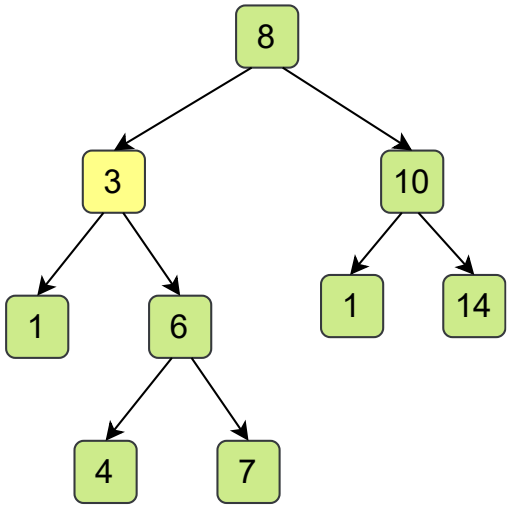
Count: 0

Result: 0



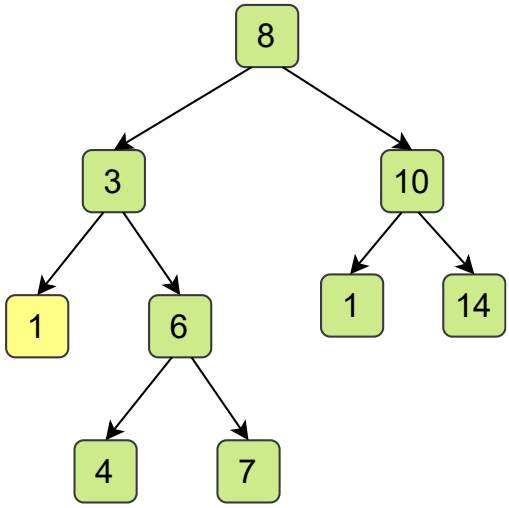
Count: 0

Result: 0



Count: 0

Result: 0



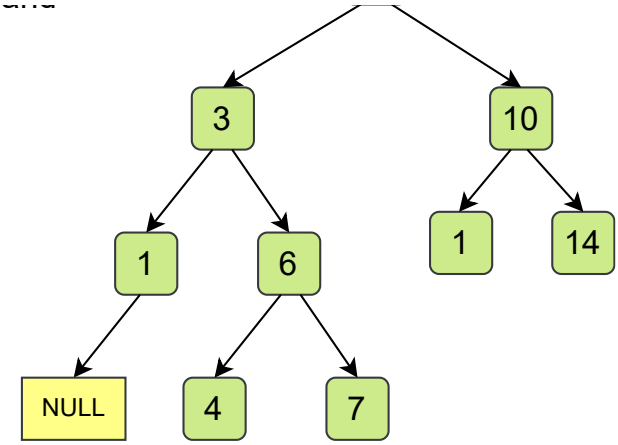
Since NULL is passed so exit condition will be called and we will simply return from here and



reach node 1

Count: 0

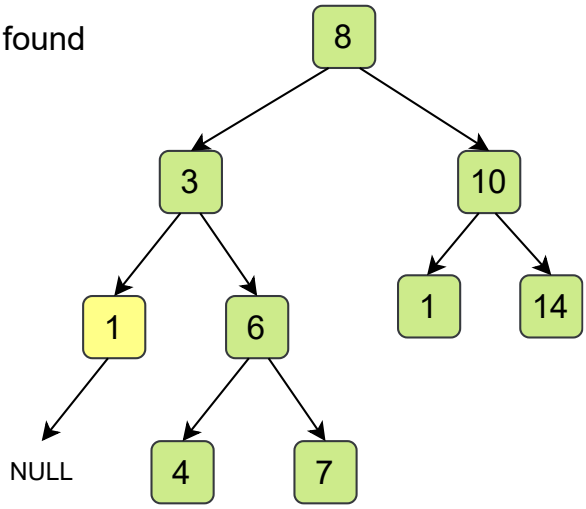
Result: 0



Now, Count will be incremented as we have found our first smallest element

Count: 1

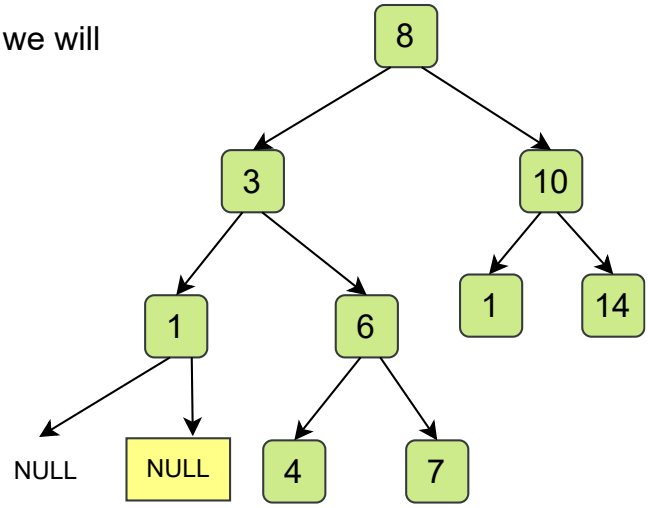
Result: 0



Now, we will move to node 1's right and we will simply returned as its NULL

Count: 1

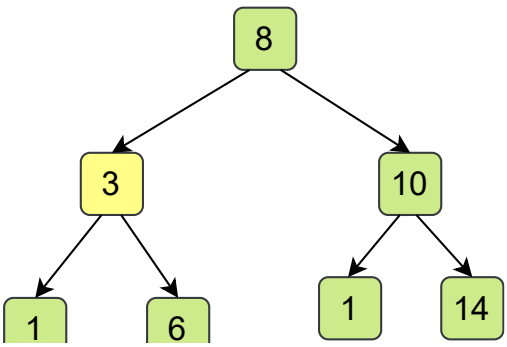
Result: 0

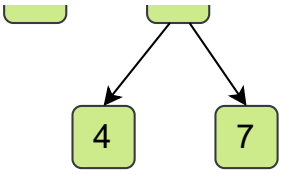


Now, we will traverse back and reach node 3. Count will be incremented as we have found our second smallest element

Count: 2

Result: 0

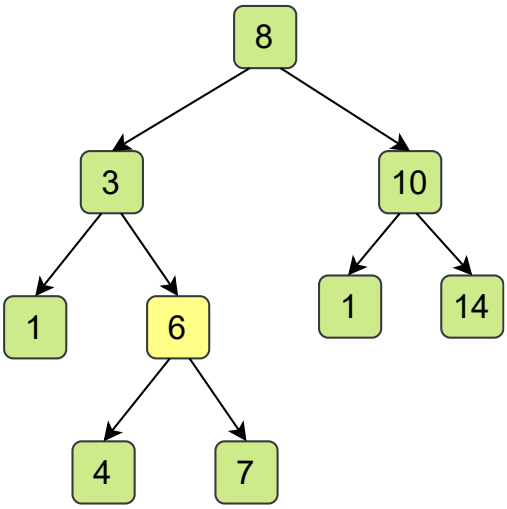




Now, we will reach node 6

Count: 2

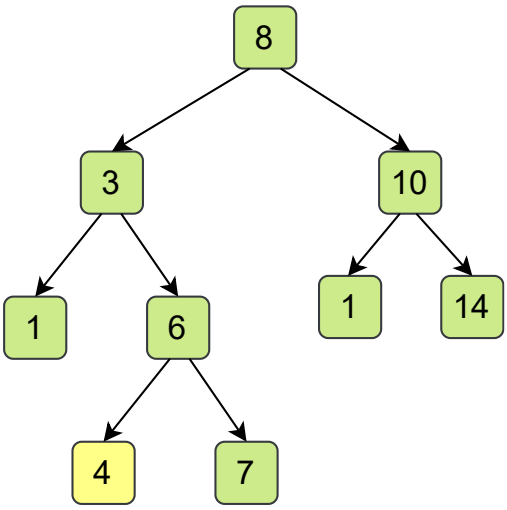
Result: 0



Now, we will move to node 6's left

Count: 2

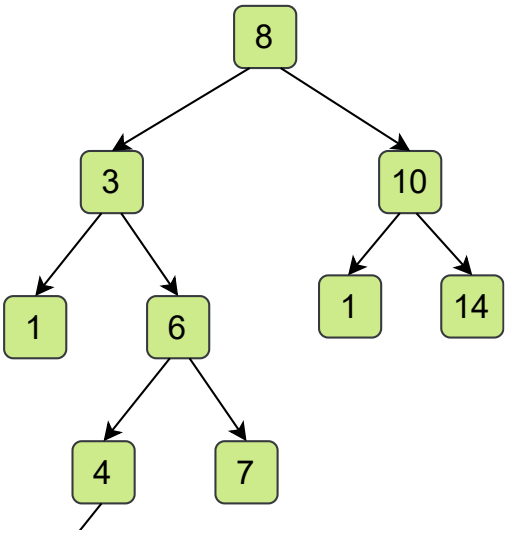
Result: 0

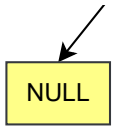


Now, we will move to node 4's left
which is NULL so we will return from here
and will traverse back to 4

Count: 2

Result: 0

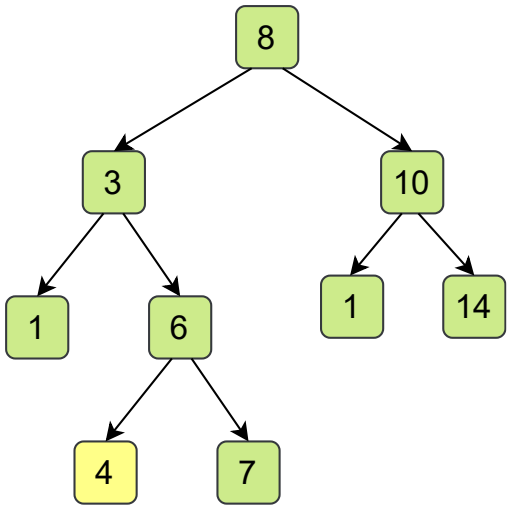




Now, 3rd smallest element has been found and count will be increased by 1

Count: 3

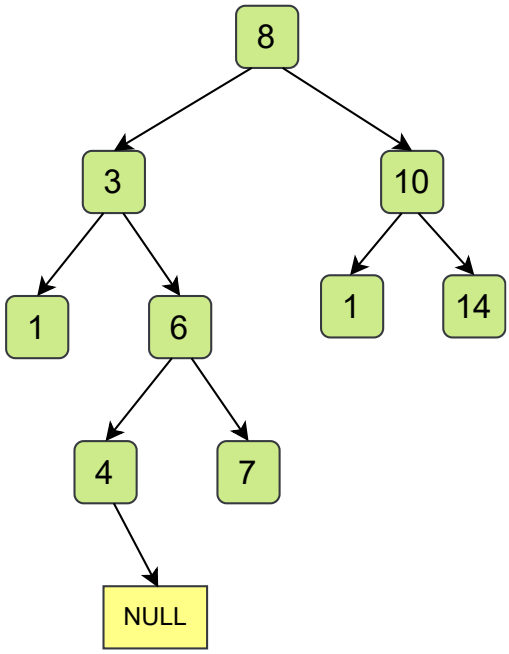
Result: 0



Now, node 4's right is NULL so we will simply return

Count: 3

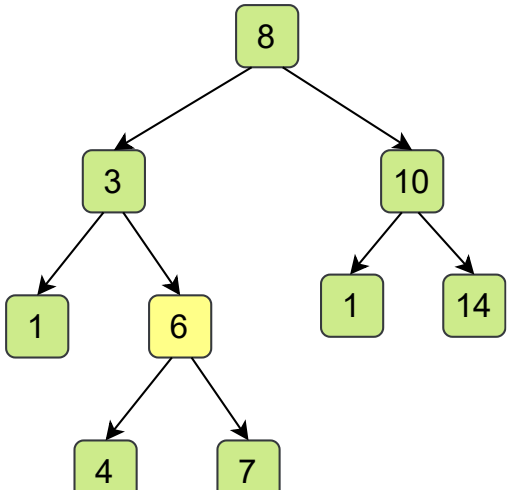
Result: 0



Now, we will traverse back to node 6 and hence count will be updated
Since Count is now equal to 4 so Result will be 6

Count: 4

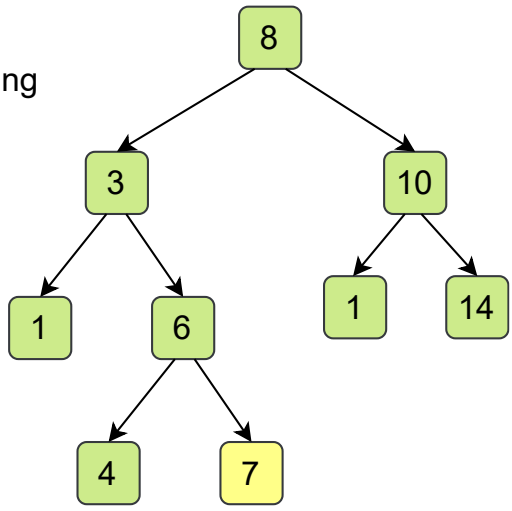
Result: 6



Now, we will move to node 6's right as we are doing inorder traversal(Left, Root, Right)

Count: 4

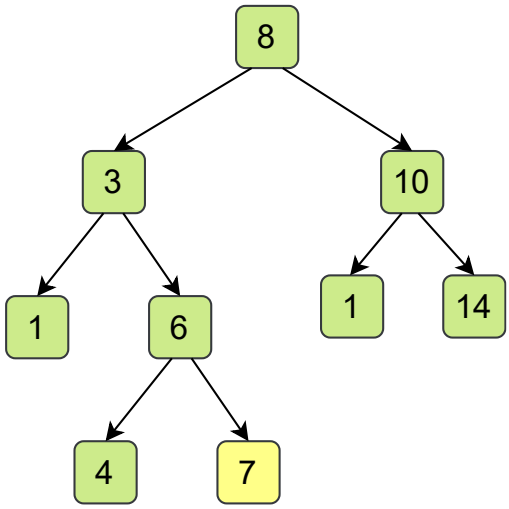
Result: 6



Since, count is 4 so we will simply return and will traverse back to 6 -> 3 -> 8. Node 8 is the position from where we started.

Count: 4

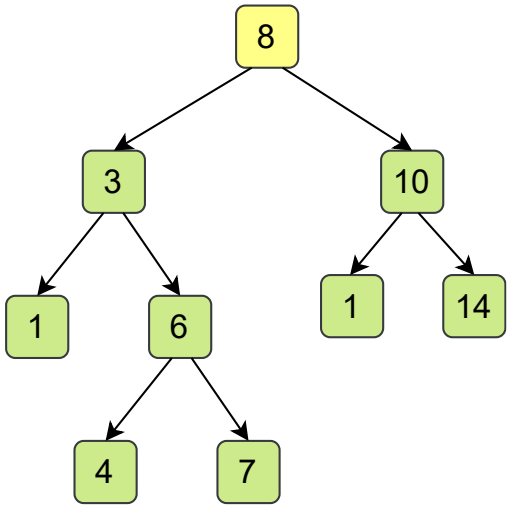
Result: 6



On reaching node 8 again, we will increment count by 1 as 7 is the 5th smallest element

Count: 5

Result: 6



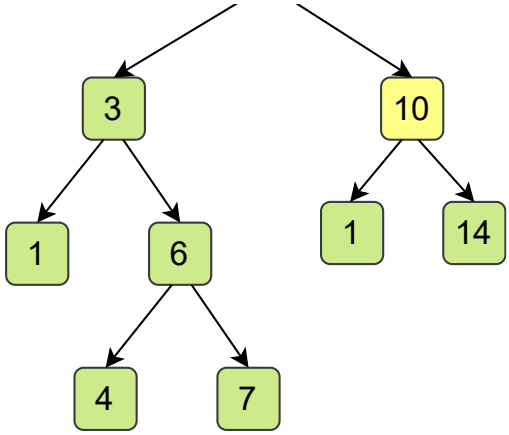
Now, we will move to node 10 but since count is >= 4 so we will simply return



so we will simply return

Count: 5

Result: 6



Result: 6

Kth Smallest Element BST

Code

Here is the code for this algorithm:

Java

```java
// class TreeNode {
//     int val;               // Value of the node.
//     TreeNode left;         // Reference to the left child.
//     TreeNode right;        // Reference to the right child.

//     // Constructor to initialize the node with its value.
//     TreeNode(int x) { val = x; }
// }

public class Solution {

    // `count` keeps track of the number of nodes we've traversed in-order.
    private int count = 0;

    // `result` will hold our final answer.
    private int result = 0;


     // This method is the public API that finds the kth smallest element in a BST.
    public int kthSmallest(TreeNode root, int k) {
        // Start the in-order traversal.
        traverse(root, k);

        // Once traversal is done, the `result` will hold our answer.
```

Run Code Result | Stdout

You must run your code first

Execute

Complexity Analysis

The worst-case time complexity is (O(N)) where (N) is the number of nodes, in case we need to visit all nodes. However, on average, we only have to visit k nodes, making it O(k). The space complexity is (O(1)) if we disregard the space used by the recursion stack, otherwise, it's (O(h)) where (h) is the height of the tree.

← Previous

Kth Smallest Element In A BST (Medium)

Next →

Closest Binary Search Tree Value (Medium)

Mark as Completed